

Contents

Option A — Manual (recommended while pre-release)	2
Option B — <code>.luarc.jsonc</code> workspace config	3
Option C — <code>luarocks</code>	3
Option D — LLS-Addons registry	4
Q: Does this work with the old <code>.conf</code> -based Hyprland config? . .	6
Q: Why is <code>HL.Vec2Like</code> / <code>HL.CssGap</code> / <code>HL.Gradient</code> undefined in <code>hl.config.lua</code> ?	6
Q: The <code>hl</code> global still shows as undefined after install.	6
Q: How do I add this to a NixOS / Home Manager setup?	6
Q: Should I use <code>config.jsonc</code> or <code>hyprlua.rockspec</code> ?	7

Qompass AI HyprLua

lua_ls addon providing full EmmyLua annotations for the Hyprland 0.55 Lua
config API

Profile views 0 downloads repo not found

<p align="center"

Qompass AI HyprLua

lua_ls addon providing full EmmyLua annotations for the Hyprland 0.55 Lua
config API

Quick Start

```
git clone https://github.com/qompassai/hyprlua \
~/.local/share/nvim/hyprlua
```

Then in your Neovim `lua_ls` setup:

```
require("lspconfig").lua_ls.setup({
  settings = {
    Lua = {
      runtime = { version = "Lua 5.4" },
      workspace = {
        library = {
          vim.fn.expand("~/local/share/nvim/hyprlua/library"),
        },
      },
      diagnostics = { globals = { "hl" } },
    },
  },
})
```

Addon Structure

hyprlua/

config.jsonc	← LLS-Addons legacy manifest
hyprlua.rockspec	← Canonical manifest (luarocks-build-lls-addon)
library/	
hl.lua	← All runtime types, classes, namespaces, hl global
hl.config.lua	← HL.ConfigKey alias (300+ keys) + HL.ConfigValueTypes
LICENSES/	
Apache-2.0.txt	← Qompass AI original work
BSD-3-Clause.txt	← Hyprland upstream stub attribution
plugin.lua	← lua_ls OnSetText hook (scoped to hypr*.lua)
README.md	
README.pdf	
settings.json	← Recommended settings for manual Neovim installs

File	Purpose
library/hl.lua	All runtime types, classes, sub-namespaces (hl.dsp.*, hl.layout.*, hl.notification.*), and the hl global typed as HL.API
library/hl.config.lua	HL.ConfigKey alias + HL.ConfigValueTypes with concrete return types per key
plugin.lua	lua_ls OnSetText hook — pass-through with optional shebang stripping, scoped to hypr*.lua URIs
hyprlua.rockspec	Authoritative addon manifest using luarocks-build-lls-addon build type
config.jsonc	Legacy config.json-style manifest kept for LLS-Addons registry compat
settings.json	Workspace settings for users who add the library path manually
LICENSES/Apache-2.0.txt	License for original Qompass AI contributions
LICENSES/BSD-3-Clause.txt	Required upstream attribution for Hyprland-derived stubs

Installation

Option A — Manual (recommended while pre-release)

```
git clone https://github.com/qompassai/hyprlua \
  ~/.local/share/nvim/hyprlua
```

Add to your lua_ls setup:

```

-- nvim-lspconfig
require("lspconfig").lua_ls.setup({
  settings = {
    Lua = {
      runtime      = { version = "Lua 5.4" },
      workspace    = { library = { vim.fn.expand("~/local/share/nvim/hyprlua/library") } },
      diagnostics = {
        globals = {
          "hl"
        }
      }
    },
  },
},
})

-- Neovim nightly native LSP (vim.lsp.config)
vim.lsp.config("lua_ls", {
  settings = {
    Lua = {
      runtime      = { version = "Lua 5.4" },
      workspace    = { library = { vim.fn.expand("~/local/share/nvim/hyprlua/library") } },
      diagnostics = { globals = { "hl" } },
    },
  },
})

```

Option B — .luarc.jsonc workspace config

Add to your Hyprland config directory's .luarc.jsonc:

```

{
  "runtime": { "version": "Lua 5.4" },
  "workspace": {
    "library": ["~/local/share/nvim/hyprlua/library"],
    "checkThirdParty": false
  },
  "diagnostics": {
    "globals": ["hl"],
    "disable": ["lowercase-global"]
  }
}

```

Option C — luarocks

luarocks install hyprlua

Requires luarocks-build-lls-addon. The library path is resolved automati-

cally.

Option D — LLS-Addons registry

Once published to the community registry, enable via:

```
{ "Lua.addonManager.enable": true }
```

Then install `hyprlua` from the addon manager UI in your editor.

Usage

Full autocomplete, hover docs, and type-checking on the `hl` global in `hyprland.lua`:

```
-- hl.get_active_window() → HL.Window/nil (typed, autocompleted)
local win = hl.get_active_window()
if win then
    hl.notification.create({ text = win.class, timeout = 3000 })
end

-- Typed bind options + dispatcher namespaces
hl.bind("SUPER + RETURN", hl.dsp.exec_cmd("kitty"), { description = "Open terminal" })

-- Custom layout - full HL.LayoutContext / HL.LayoutTarget types
hl.layout.register("columns", {
    recalculate = function(ctx)
        local n = #ctx.targets
        for i, t in ipairs(ctx.targets) do
            t:place(ctx.column(i, n))
        end
    end,
})

-- HL.EventName alias - all 28 event strings checked + autocompleted
hl.on("window.open", function(win)
    if win.class == "steam" then
        hl.dispatch(hl.dsp.window.float())
    end
end)

-- HL.ConfigKey alias - 300+ dotted keys, narrowed return types
local rounding = hl.get_config("decoration.rounding") -- → integer/boolean
```

What you get:

- Autocomplete on all `hl.*` functions and sub-namespaces (`hl.dsp.*`, `hl.notification.*`, `hl.layout.*`, `hl.plugin.*`)
- `HL.EventName` alias — all 28 event strings are checked and completed

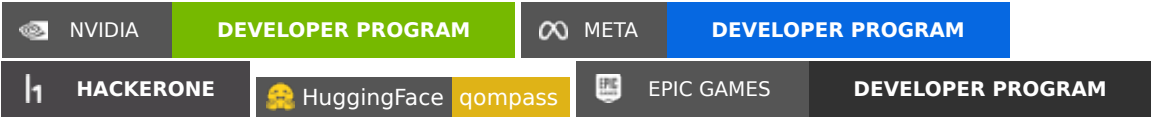
- `HL.ConfigKey` alias — 300+ dotted config keys with narrowed return types via `HL.ConfigValueTypes`
- Typed classes: `HL.Window`, `HL.Workspace`, `HL.Monitor`, `HL.Keybind`, `HL.Timer`, `HL.Notification`, `HL.Group`
- Hover docs on every method, field, and option table

About Qompass AI

Matthew A. Porter Former Intelligence Officer Educator & Learner DeepTech
Founder & CEO

Publications

Developer Programs



Professional Profiles

Social Media

How Do I Support

Qompass AI Pre-Seed Funding 2023-2025

Amount

Date

RJOS/Zimmer Biomet Research Grant

\$30,000

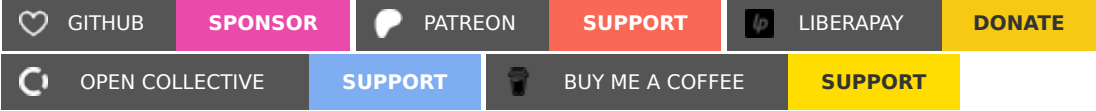
March 2024

Pathfinders Intern Program View on LinkedIn

\$2,000

October 2024

How To Support Our Mission



Cryptocurrency Donations

Monero (XMR):

`<code>42HGspSFJQ4MjM5ZusAiKZj9JZWhfNgVraKb1eGCsHoC6QJqpo2ERCBZDhhKfByVjECernQ6KeZwFcnq8hVwT`

Copy Address

Funding helps us continue our research at the intersection of AI, healthcare, and education

What a Dual-License Means

This addon uses a **dual-license** model:

- **Apache 2.0** — applies to all original Qompass AI work: Emmy-Lua/LuaCATS annotations, doc comments, addon structure, Neovim integration, and the rockspec.
- **BSD 3-Clause** — applies to stub definitions derived from the Hyprland source generator (`scripts/generateLuaStubs.py`). This attribution is required and cannot be removed.

Apache 2.0 and BSD 3-Clause are compatible licenses. You may use, modify, and redistribute this addon under either license provided the BSD attribution block is preserved in any file that incorporates upstream-derived stubs.

Full license texts are in `LICENSES/Apache-2.0.txt` and `LICENSES/BSD-3-Clause.txt`.

Frequently Asked Questions

Q: Does this work with the old `.conf`-based Hyprland config?

A: No. This addon targets the **Lua config API** introduced in Hyprland 0.55. The old `hyprls` LSP handled `.conf` files and is now largely obsolete. If you are still on `.conf`, no action is needed here.

Q: Why is `HL.Vec2Like` / `HL.CssGap` / `HL.Gradient` undefined in `hl.config.lua`?

A: These aliases are defined in `hl.lua`. Both files must be in the same `workspace.library` path. The aliases are also re-declared at the top of `hl.config.lua` as a fallback for `lua_ls` versions that don't resolve cross-file `@meta` aliases automatically.

Q: The `hl` global still shows as undefined after install.

A: Verify `diagnostics.globals = { "hl" }` is set in your `lua_ls` settings, or that your `.luarc.jsonc` in the Hyprland config directory includes `"diagnostics": { "globals": ["hl"] }`. You can also check `:LspLog` in Neovim for any path resolution errors.

Q: How do I add this to a NixOS / Home Manager setup?

A: Point `programs.neovim.extraLuaConfig` or your `lua_ls workspace.library` at a `pkgs.fetchFromGitHub` derivation targeting this repo. A `flake.nix` is planned for a future release.

Q: Should I use `config.jsonc` or `hyprlua.rockspec`?

A: The `*.rockspec` is the current canonical manifest used by `luarocks-build-lls-addon` and the LLS addon manager. `config.jsonc` is kept for legacy compatibility with older LLS-Addons tooling. If you are installing manually via `workspace.library`, neither file is required on your end.